BLACK MARKET COALITION

Unified App Build Plan

Spatial-First Social Platform | Solarpunk Design | Consent-Based | Federated

Cross-Platform (iOS + Android + Web) | MapLibre GL + Self-Hosted Tiles
React Native + Tamagui | Matrix E2EE | MedusaJS + Fleetbase + Synapse

March 2026

Contents

Phase 0: Repository Setup & Foundation Architecture (Weeks 1-2)
Phase 1: Map + Marketplace Browsing (Weeks 3-6)
Phase 2: Blackout Video Feed + Chat (Weeks 7-10)
Phase 3: Jobs & Logistics Integration (Weeks 11-14)
Phase 4: Governance & Voting (Weeks 15-18)
Phase 5: Mutual Aid & Community Layer (Weeks 19-22)
Phase 6: Polish, Consent Flows & App Store (Weeks 23-26)
AI Prompt Reference Index

Architecture Overview

The BMC App is a spatial-first social platform that unifies the Black Market Coalition ecosystem: freeblackmarket.com (marketplace), Blackstar (federated logistics), and Blackout (Matrix-based encrypted messaging and governance). The interface centers on a map where every feature in the ecosystem appears as a tappable icon. Users can browse offerings, watch a TikTok-style video feed with E2EE chatroom comments, claim delivery jobs, vote on governance proposals, and coordinate mutual aid — all from a single app that works on iOS, Android, and web.

The app is forked from blackstar_nav (the existing Fleetbase driver app) which already provides React Native 0.77, React Native Web, Tamagui UI framework, map components, SocketCluster real-time, auth flows, chat, location tracking, and navigation. This gives a massive head start over building from scratch.

Ecosystem Integration Map

System	Stack	Role in App	Existing Bridge
FreeBlackMarket	MedusaJS / Node / Postgres	Marketplace, gardens, campaigns, time bank	REST API + JS SDK
Blackstar	Laravel / PHP / MySQL	Jobs, delivery, logistics nodes, driver tracking	FBM Webhook Controller
Blackout Server	Synapse / Python / Postgres	E2EE messaging, governance rooms, video feed	Matrix Client-Server API
Blackout (Element)	React / TypeScript	Governance engines, steganography, proposals	Feature flag modules
Blackstar Nav	React Native / Tamagui	BASE FORK: maps, auth, chat, orders, location	Already built

Key Design Principles

- General user first: the app works for customers, browsers, and community members — not just vendors and drivers
- Consent-based location: approximate location only, never exact address, always opt-in with clear privacy controls
- Solarpunk aesthetic: Fibonacci spirals, sine waves, light rays, organic blob shapes, earth-tone palette
- Social media UX patterns: Snap Map-style icon exploration, TikTok-style vertical video feed, Instagram-style profiles
- Blackout-native: all chat and comments flow through Matrix E2EE rooms; video feed comments are live chatroom conversations
- Offline-capable: PMTiles for map, SQLite for data cache, Yjs CRDT for collaborative state, action queue for connectivity gaps

PHASE 0 Weeks 1–2

Repository Setup & Foundation Architecture

Fork blackstar_nav, restructure as a monorepo, replace the map engine, and deploy the tile server. This phase produces a running app shell with a real self-hosted map and no feature logic yet.

Tasks

#	Task	Owner	Status
1	Fork blackstar_nav → bmc-app, restructure into Turborepo monorepo (apps/mobile, apps/web, apps/gateway, packages/ui, packages/core)	Dev	**Critical**
2	Strip Fleetbase-specific screens (DriverDashboard, FuelReport, Fleet, Vehicle); keep map, auth, chat, location, and order infrastructure	Dev	**Critical**
3	Replace react-native-maps with @maplibre/maplibre-react-native; update PlaceMapView, LiveOrderRoute, LocationMarker components	Dev	**Critical**
4	Deploy Martin tile server on Railway connected to FBM Postgres; add PostGIS extension; import OSM data with osm2pgsql for Sacramento region	DevOps	**High**
5	Create custom MapLibre style JSON (dark solarpunk base, teal roads, organic water glow, semi-transparent buildings)	Design	**High**
6	Set up Hono API gateway service on Railway with health check and CORS; create /api/v1/auth/unified endpoint stub	Dev	**High**
7	Design Adinkra-inspired SVG icon sprite set for all entity types (vendor, job, garden, vote, aid, infra) as MapLibre sprite sheet	Design	**High**
8	Configure GitHub Actions CI: build iOS, Android, Web; run ESLint + TypeScript checks	DevOps	**High**
9	Generate PMTiles archive for Sacramento metro area; configure offline tile loading in MapLibre	Dev	**Normal**

AI Prompts

AI PROMPT — Monorepo Scaffold

> *I have a React Native app called blackstar_nav (React Native 0.77 + Tamagui + react-native-maps + React Navigation v7). I need to restructure it into a Turborepo monorepo with this structure: apps/mobile (the RN app), apps/web (React Native Web build), apps/gateway (Hono API server), packages/ui (shared Tamagui components), packages/core (shared business logic). The existing app*

> *uses: AuthContext, LocationContext, ChatContext, OrderManagerContext, SocketClusterProvider, ThemeContext, ConfigProvider. Keep all these contexts in packages/core. Move all UI components (MapView, BottomSheet, cards) into packages/ui. Generate the turborepo.json, root package.json, and updated imports. The app currently has these screens I want to KEEP: BootScreen, LoginScreen, OrderScreen, ChatHomeScreen, ChatChannelScreen, EntityScreen. REMOVE all driver-specific screens: DriverDashboardScreen, DriverFleetScreen, FuelReportScreen, VehicleScreen, DriverReportScreen. Show me the complete file restructure.*

AI PROMPT — MapLibre Replacement

> *Replace react-native-maps with @maplibre/maplibre-react-native in my React Native 0.77 app. I have these existing components that use MapView and Marker from react-native-maps: PlaceMapView.tsx (shows a single location marker), LiveOrderRoute.tsx (shows a route polyline between waypoints with driver tracking), LocationMarker.tsx, DriverMarker.tsx, TrackingMarker.tsx. The new map should: (1) Load vector tiles from my self-hosted Martin tile server at MARTIN_TILE_URL env var, (2) Use a custom style JSON for a dark solarpunk theme, (3) Support clustering with maplibre-gl cluster properties, (4) Support custom sprite icons loaded from a sprite sheet URL. Show me the updated components using MapLibre's Camera, ShapeSource, SymbolLayer, CircleLayer, and LineLayer instead of the react-native-maps equivalents. Include the Podfile and build.gradle changes needed.*

AI PROMPT — Martin Tile Server + PostGIS Setup

> *I need to deploy a Martin tile server on Railway connected to my existing PostgreSQL database (which runs MedusaJS for freeblackmarket.com). Steps needed: (1) Add PostGIS extension to the existing Postgres instance, (2) Import OpenStreetMap data for the Sacramento, California metro area using osm2pgsql with the flex output, (3) Configure Martin to serve vector tiles from the imported OSM tables AND from custom application tables (I have vendor locations, garden plots, delivery zones stored in my Medusa database), (4) Create a Dockerfile for Martin that connects to the DATABASE_URL env var, (5) Generate a PMTiles archive of the Sacramento area for offline use. Show me the martin.yaml config, Dockerfile, osm2pgsql flex style lua file, and the Railway deployment config. My Postgres connection string format is postgresql://user:pass@host:port/dbname.*

AI PROMPT — Dark Solarpunk MapLibre Style

> *Create a MapLibre GL style JSON for a dark solarpunk map theme. Requirements: (1) Background color #0a1a0f (deep forest black), (2) Roads as thin lines in #4ade80 (leaf green) with major roads slightly thicker and brighter, (3) Buildings as semi-transparent polygons in rgba(74,222,128,0.08) with subtle green borders, (4) Water bodies with a gradient glow effect using fill-color rgba(34,211,238,0.15) and a halo, (5) Parks and green spaces in rgba(74,222,128,0.12), (6) Labels in #a3c9ab (muted green) using Nunito Sans font, (7) Contour lines as very subtle sine-wave-like strokes if topography data is available, (8) No highway shields or transit symbols — keep it minimal and organic. The tile source is my Martin server at {martin_url}/tiles. Include the sprite URL pattern for custom icons.*

PHASE 1 Weeks 3–6

Map + Marketplace Browsing

Wire the map to real data from freeblackmarket.com. Users can browse vendors, view products, see gardens and kitchens as map icons, and place orders. This is the core loop that makes the app immediately useful for general users.

Tasks

#	Task	Owner	Status
1	Build gateway /api/v1/spatial/feed endpoint: query MedusaJS for vendors, products, gardens, kitchens, producers with location data; return as GeoJSON FeatureCollection	Dev	**Critical**
2	Build MapCanvas component with ShapeSource + SymbolLayer rendering GeoJSON feed; implement clustering at zoom levels	Dev	**Critical**
3	Build category filter chips (horizontal scroll): Grower, Maker, Mover, Healer, Teacher, Builder, Organizer; filter map features by category	Dev	**High**
4	Build ProfileSheet bottom sheet: entity preview on icon tap with avatar, bio, stats (Phi-proportioned), offerings tab, activity tab	Dev	**High**
5	Build Home tab: greeting, quick action grid (Shop Local, Watch Feed, Mutual Aid, Vote), trending content, nearby scroll (location-gated)	Dev	**High**
6	Implement unified auth: gateway JWT wrapping medusa_customer_id + roles; login/register screens adapted from existing AuthContext	Dev	**Critical**
7	Build product browsing flow: tap vendor → see offerings → add to cart → checkout (embed MedusaJS storefront cart/checkout logic)	Dev	**High**
8	Add location data to MedusaJS vendor/garden/kitchen models via seller-extension module; store as PostGIS geometry columns	Dev	**High**
9	Build LocationConsent dialog with approximate-only explanation, privacy guarantees, and Skip option; persist preference in AsyncStorage	Dev	**High**
10	Implement Search tab: text search across vendors/products/gardens via gateway; results as list with tap-to-map navigation	Dev	**Normal**

AI Prompts

AI PROMPT — Gateway Spatial Feed

> *Build a Hono API endpoint at GET /api/v1/spatial/feed that: (1) Accepts query params: bbox (west,south,east,north), layers (comma-separated: market,jobs,govern,aid), limit, offset. (2) Queries my MedusaJS backend at MEDUSA_BACKEND_URL using the @medusajs/js-sdk to fetch: sellers (with store name, handle, description, location), products (with inventory, pricing), gardens (from custom garden module), kitchens (from custom kitchen module), producers (from custom producer module). (3) Each entity needs location coordinates — these come from a custom seller_location table I'll add with PostGIS point geometry. (4) Returns a GeoJSON FeatureCollection where each Feature has: geometry (Point), properties (layer, entity_type, id, name, status, icon sprite name, preview object with image_url/tagline/rating). (5) Filters by bounding box using PostGIS ST_Within. (6) Uses Redis cache with 60s TTL for the spatial queries. Show me the complete Hono route, the MedusaJS service calls, and the GeoJSON transformation. TypeScript with full types.*

AI PROMPT — MapCanvas + Clustering Component

> *Build a React Native component called MapCanvas using @maplibre/maplibre-react-native that: (1) Loads tiles from my Martin server (MARTIN_TILE_URL env), (2) Accepts a GeoJSON FeatureCollection as a features prop, (3) Renders features using ShapeSource with cluster=true, clusterMaxZoomLevel=14, clusterRadius=50, (4) Unclustered points render as SymbolLayer using icon-image from the feature's properties.icon field (mapped to my sprite sheet), (5) Clustered points render as CircleLayer with size proportional to point_count, colored by the dominant category in the cluster, (6) On icon tap: calls onEntitySelect(feature) prop, (7) On cluster tap: zooms into the cluster bounds, (8) Has a ref-exposed method flyTo(coordinates, zoom) for programmatic navigation, (9) Shows the user's location as a pulsing green dot (only if location permission granted). Use Tamagui for any overlay UI. The component should work on both RN and RN Web via MapLibre GL JS on web.*

AI PROMPT — MedusaJS Location Extension

> *I need to add location data to my MedusaJS v2 + MercurJS multi-vendor marketplace. Create a Medusa module called seller-location that: (1) Adds a seller_location model with fields: seller_id (links to MercurJS seller), coordinates (PostGIS point), address_line, city, state, zip, country, display_radius (how approximate to show on map — for privacy), is_visible (boolean — consent toggle), location_type (enum: storefront, farm, kitchen, garden, mobile, online_only). (2) Creates a link between the seller_location module and the existing seller module. (3) Exposes API routes: POST /vendor/seller-location (create/update own location, requires vendor auth), GET /store/seller-locations (public, returns approximate locations within bbox, fuzzed by display_radius), DELETE /vendor/seller-location (remove own location). (4) The GET endpoint should fuzz coordinates by adding random offset within the display_radius before returning — never expose exact coordinates. (5) Include the MikroORM migration to add the PostGIS point column. Show me the full module: index.ts, models, service, API routes, and migration.*

AI PROMPT — Location Consent Flow

> *Build a React Native LocationConsent component using Tamagui that: (1) Shows a modal dialog explaining location sharing with these guarantees: approximate only (never exact address), visible only to people you choose, hideable anytime, pauses when app is closed. (2) Has two buttons: 'Allow Approximate Location' (calls react-native-permissions to request location, then stores*

consent in AsyncStorage as 'location_consent': 'granted') and 'Not Now' (stores 'denied', app continues without location features). (3) A Privacy section in settings where users can revoke consent at any time (sets 'revoked' and calls the seller-location DELETE endpoint if they had shared). (4) The consent state is exposed via a LocationConsentContext with: consentStatus, requestConsent(), revokeConsent(), and isLocationAvailable boolean. (5) All map features gracefully degrade when location is denied — show a centered prompt instead of markers. (6) Location is fuzzed client-side before any API call: add random offset within 0.2 mile radius. Use the existing LocationContext and LocationProvider from blackstar_nav as the base.

PHASE 2 Weeks 7–10

Blackout Video Feed + Chat

Build the TikTok-style vertical video feed where each video is tied to a Blackout Matrix room. Comments are live E2EE chatroom conversations. This is the social hook that drives engagement for all users, not just buyers.

Tasks

#	Task	Owner	Status
1	Integrate matrix-js-sdk into the app; create MatrixContext providing login, sync, room list, and event listener	Dev	Critical
2	Build VideoFeed screen: full-screen vertical swipe (FlatList with pagingEnabled), video playback via react-native-video, swipe up/down navigation	Dev	Critical
3	Build video action sidebar: like (heart), comment (opens chat panel), share, find-on-map (navigates to creator's map marker), follow	Dev	High
4	Build ChatPanel bottom sheet for video comments: renders Matrix room timeline as comment stream; E2EE via matrix-js-sdk Crypto; input sends m.room.message events	Dev	Critical
5	Create video upload flow: record/select video → compress → upload to Matrix media repo (or MinIO) → create m.room.message with video attachment in creator's feed room	Dev	High
6	Build Chat tab: room list from Matrix sync, DMs and group rooms, unread badges, E2E encrypted indicator per room	Dev	High
7	Wire unified auth to Matrix: gateway creates Matrix account on registration, returns access_token + device_id alongside gateway JWT	Dev	Critical
8	Build video feed room structure: each creator gets a !feed:{homeserver} room; videos are m.room.message events with m.video msgtype; comments are threaded replies	Dev	High
9	Implement Blackout room badge on each video showing the room ID; tap to open full chatroom	Dev	Normal
10	Add Feed tab to tab bar as raised center button; tapping opens full-screen VideoFeed overlay	Dev	Normal

AI Prompts

AI PROMPT — Matrix SDK Integration

> *Integrate matrix-js-sdk into my React Native 0.77 + Tamagui app for the Blackout messaging layer. Build a MatrixContext provider that: (1) On app*

> *launch, checks AsyncStorage for matrix_access_token and matrix_user_id, (2) If tokens exist, creates a MatrixClient, starts sync with a filter for relevant rooms only (tagged with m.bmc.feed, m.bmc.governance, m.bmc.chat), (3) Exposes: client instance, isLoggedIn, rooms (sorted by recent activity), sendMessage(roomId, content), createRoom(options), joinRoom(roomIdOrAlias), getTimeline(roomId, limit), (4) Handles E2EE setup: initialize crypto on first login, request/share room keys, verify devices. (5) Login flow: receives matrix credentials from the gateway auth endpoint (the gateway registers/logs in the user on the Blackout Synapse server and returns tokens). (6) Provides a useRoom(roomId) hook that returns: timeline events, members, room name, typing indicators, and a send function. The Synapse homeserver URL comes from BLACKOUT_HOMESERVER_URL env var. Use @matrix-org/matrix-sdk-crypto-wasm for crypto. Show the full context, provider, and hooks.*

AI PROMPT — TikTok Video Feed Component

> *Build a TikTok-style vertical video feed in React Native using react-native-video and a FlatList with pagingEnabled and snapToInterval. Requirements: (1) Each video fills the entire screen (100% width, 100% height of the container), (2) Videos auto-play when they become the active item and pause when scrolled away (use onViewableItemsChanged with viewabilityConfig threshold 0.7), (3) Right sidebar with: creator avatar (organic blob border-radius, tap to view profile), like button with count (animated heart on tap using react-native-reanimated), comment button with count (opens ChatPanel), share button, and map pin button (navigates to creator on map), (4) Bottom overlay with: creator name + handle, caption text (max 3 lines, expandable on tap), and a Blackout room badge showing the Matrix room ID, (5) Swipe indicators on the right edge showing position in feed, (6) Double-tap to like (with heart animation burst), (7) Long-press to share options. Videos are loaded from Matrix room events: each event has content.url pointing to the video file on the Matrix media server. The feed queries a gateway endpoint GET /api/v1/feed that returns videos sorted by engagement. Style with Tamagui and the solarpunk palette: dark backgrounds, organic shapes, green/gold accents.*

AI PROMPT — E2EE Chat Comments Panel

> *Build a ChatPanel component that renders Matrix room comments for the Blackout video feed. Requirements: (1) Slides up from the bottom as a half-screen panel (like TikTok comments) using react-native-reanimated and @gorhom/bottom-sheet, (2) Receives a roomId prop and uses the useRoom(roomId) hook from MatrixContext, (3) Renders the room timeline as a scrollable comment list: each comment shows avatar (organic blob shape), username, message text, and relative timestamp, (4) Shows an 'E2E Encrypted' badge in the header, (5) Has a text input at the bottom that sends m.room.message events to the room, (6) New messages appear at the bottom with a slide-in animation, (7) Shows typing indicators when other users are composing, (8) Handles encrypted events: decrypt using the MatrixClient crypto, show a lock icon for verified senders, (9) The panel background uses the solarpunk surface glass: rgba(18,42,24,0.88) with backdrop-filter blur. This panel is used in two contexts: overlaid on the video feed, and as the full Chat tab room view. Make it reusable. Style everything with Tamagui.*

PHASE 3 Weeks 11–14

Jobs & Logistics Integration

Wire Blackstar's federated logistics into the map. Users can see open delivery jobs, farm work, and volunteer opportunities as map icons. Drivers/workers can claim jobs and navigate to them. Orders placed in Phase 1 generate shipment listings that appear as claimable jobs.

Tasks

#	Task	Owner	Status
1	Add Blackstar API client to gateway: fetch shipment board listings, eligible jobs, node trust scores, active deliveries	Dev	**Critical**
2	Extend /spatial/feed to include jobs layer: open shipment listings with origin/destination coordinates, job type, payout estimate	Dev	**High**
3	Build JobSheet bottom sheet: job details, claim/bid buttons, route preview, payout info, required capabilities	Dev	**High**
4	Build NavigationView: turn-by-turn directions using ArcGIS Route API (already in LiveOrderRoute.tsx); sine-wave path visualization, ETA, destination info	Dev	**High**
5	Wire FBM order → Blackstar shipment: when an order is placed, the gateway creates a ShipmentBoardListing via Blackstar API with source_order_ref	Dev	**Critical**
6	Build role-based visibility: drivers see 'Claim Job' buttons; buyers see delivery tracking; role comes from gateway JWT	Dev	**High**
7	Implement real-time delivery tracking via SocketCluster (already in blackstar_nav): show moving driver icon on map for active deliveries	Dev	**Normal**
8	Add provider profile creation flow: users opt-in to become a Grower/Maker/Mover/etc; this creates accounts in Blackstar (node + driver) and MedusaJS (seller)	Dev	**High**

AI Prompts

AI PROMPT — Blackstar Gateway Client

> *Build a TypeScript API client for my Blackstar server (Fleetbase fork, Laravel REST API) to use in my Hono gateway. The Blackstar API has these endpoints: GET /api/v1/shipment-board-listings (list open shipments), GET /api/v1/shipment-board-listings/eligible (filtered by authenticated node's capabilities), POST /api/v1/shipment-board-listings (create listing), POST /api/v1/shipment-board-listings/{id}/claim (claim a job), POST /api/v1/shipment-board-listings/{id}/bid (place a bid), GET /api/v1/nodes (list nodes with trust scores), GET /api/v1/drivers (list drivers), GET /api/v1/fleets (list fleets). Auth is Bearer token. Each listing has:*

> *source_order_ref, origin, destination, claim_policy (first_claim or bid), required_category, required_transport_capabilities, status. Build the client class with full TypeScript types, error handling, and a method to transform listings into GeoJSON Features (using origin/destination as Point geometries) for the spatial feed. Include the FreeBlackMarket webhook integration: when an order is placed on FBM, create a shipment listing on Blackstar with the correlation_id pattern from the existing FreeBlackMarketWebhookController.*

AI PROMPT — Provider Profile Onboarding

> *Build a multi-step provider profile creation flow in React Native + Tamagui. This upgrades a general user to a provider (vendor/driver/organizer). Steps: (1) Role selection: grid of roles with Adinkra-inspired icons (Grower, Maker, Mover, Healer, Teacher, Builder, Organizer) — multi-select, (2) Profile details: name, handle, bio, avatar upload, (3) Location consent: explain that providers show on the map, let them set display_radius for privacy fuzzing (choices: Neighborhood ~0.5mi, District ~1mi, City ~3mi, Region ~10mi), (4) Offerings: add initial offerings (name, description, price or 'time bank' or 'free' or 'sliding scale'), (5) Review and confirm. On submit, the gateway: creates a MedusaJS seller account, creates a Blackstar node (if Mover role selected) + driver registration, creates a Matrix room for the provider's feed, and updates the JWT with new roles. Use Fibonacci-proportioned layouts, organic border-radius, and the solarpunk color palette. Include step indicator with golden-ratio segment widths. Animate transitions with react-native-reanimated spring physics.*

PHASE 4 Weeks 15–18

Governance & Voting

Port the governance engines from the Blackout Element fork into the app. Active proposals appear as map icons tied to geographic communities. Users can vote, delegate, and participate in townhalls directly from the app.

Tasks

#	Task	Owner	Status
1	Port ProposalEngine, VotingEngine, DelegatedVotingEngine from blackout/src/services/governance/ into packages/core; adapt from Element's React to RN-compatible logic	Dev	**Critical**
2	Build GovernanceLayer for map: proposals with location metadata rendered as vote icons; urgency pulse for closing-soon votes	Dev	**High**
3	Build ProposalSheet: proposal title, description, current vote tally, discussion thread (Matrix room), vote buttons (Approve/Reject/Abstain)	Dev	**High**
4	Build ProposalComposer: create new proposals with title, body, options, location tag, closing date; stored as Matrix room state events	Dev	**High**
5	Implement delegation: users can delegate voting power to trusted community members; port DelegatedVotingEngine logic	Dev	**Normal**
6	Build Townhall integration: LiveKit SFU video rooms appearing as map markers; tap to join audio/video townhall	Dev	**High**
7	Add governance_room_id to Blackstar nodes (already exists in model); show node governance room in the app when viewing logistics entities	Dev	**Normal**
8	Wire deliberation clustering from blackout's clusterOpinions() for large votes; show opinion clusters in ProposalSheet	Dev	**Normal**

AI Prompts

AI PROMPT — Port Governance Engines

> *I have three governance engine classes from my Element Web fork (TypeScript, browser environment) that I need to port to React Native. The source files are: (1) ProposalEngine.ts — handles proposal lifecycle: draft → discuss → amend → close → decide, stores proposals as Yjs CRDT documents synced via Matrix room state events (im.blackout.governance.proposal), (2) VotingEngine.ts — handles ballot casting: approve/reject/abstain, tallying,*

> *quorum checking, stores votes as encrypted Matrix events, (3) DelegatedVotingEngine.ts — handles vote delegation chains with attestation verification. Port these to work in React Native with these changes: (a) Replace browser IndexedDB with op-sqlite for local state, (b) Replace Element's SettingsStore references with a simple AsyncStorage-backed config, (c) Keep the Matrix event types and room state integration identical (they talk to the same Blackout Synapse server), (d) Wrap in React hooks: useProposals(roomId), useVoting(proposalId), useDelegation(userId), (e) Create a GovernanceContext provider that initializes the engines with the MatrixClient from MatrixContext. The Yjs documents should sync via y-matrix (Yjs provider that uses Matrix room events as transport). Show me the ported code with full RN compatibility.*

AI PROMPT — Governance Map Layer

> *Add a governance layer to my MapCanvas component. Governance proposals are stored as Matrix room state events (im.blackout.governance.proposal) in rooms tagged with m.bmc.governance. Each proposal can have optional location metadata (coordinates + radius). Build: (1) A gateway endpoint GET /api/v1/spatial/governance that queries Matrix rooms for active proposals with location data and returns them as GeoJSON Features with properties: proposal_id, title, status (draft/discuss/amend/close/decide), vote_tally ({approve, reject, abstain}), closes_at, room_id, (2) MapCanvas renders these as a distinct SymbolLayer with vote icons that pulse when the vote is closing within 48 hours, (3) ProposalSheet bottom sheet: shows the proposal title, description (Markdown rendered), live vote bar chart, discussion thread from the Matrix room (reuse ChatPanel), and vote buttons that call VotingEngine.castBallot(). (4) The vote buttons are: green 'Approve', red 'Reject', gray 'Abstain' with organic blob shapes. After voting, the button transforms to show the user's choice with a confirmation animation.*

PHASE 5 Weeks 19–22

Mutual Aid & Community Layer

Build the mutual aid system: community members post needs and offers with location context. Integrate time banking from the hawala-ledger module, harvest allocation from the garden module, and collective campaigns. This completes the community layer that makes the app a true mutual aid coordination platform.

Tasks

#	Task	Owner	Status
1	Build MutualAid module in FreeBlackMarket: need/offer model with location, category (food, transport, labor, materials, care), expiry, fulfillment status	Dev	**High**
2	Add mutual aid to spatial feed: needs as orange markers, offers as green markers; proximity matching (show nearby unfulfilled needs)	Dev	**High**
3	Build AidSheet: need/offer details, respond button (creates DM or joins coordination room), fulfillment confirmation flow	Dev	**High**
4	Build Post tab: create new content — listing, mutual aid request, job posting, governance proposal, campaign; each type routes to the appropriate creation flow	Dev	**High**
5	Integrate hawala-ledger: show time bank balance in profile, enable time-bank payments for services, log volunteer hours	Dev	**Normal**
6	Integrate garden module: show gardens on map, plot availability, volunteer schedules, harvest alerts, seasonal calendars	Dev	**Normal**
7	Integrate collective-campaign module: campaign progress on map (productive assets being funded), backer count, investment flow	Dev	**Normal**
8	Build notification system: SSE from gateway aggregating events across all backends; push notifications via Firebase/APNs	Dev	**High**

AI Prompts

AI PROMPT — Mutual Aid Module

> *Create a MedusaJS v2 module called mutual-aid for my freeblackmarket.com backend. Models: (1) AidPost — fields: id, customer_id (poster), type (enum: need, offer), category (enum: food, transport, labor, materials, care, housing, childcare, eldercare, tech_support, other), title, description, location (PostGIS point, fuzzy — use the display_radius pattern from seller-location), urgency (low, medium, high, critical), expires_at, status (open,*

> *in_progress, fulfilled, expired, cancelled), fulfiller_id (nullable), fulfilled_at, metadata (JSON). (2) AidResponse — fields: id, aid_post_id, responder_id, message, status (pending, accepted, declined), matrix_room_id (auto-created DM room for coordination). Service methods: createPost(), respondToPost() (creates Matrix room between poster and responder via gateway), fulfillPost(), getNearbyPosts(lat, lng, radiusKm, filters), matchNeeds() (find unfulfilled needs near the user's offers). API routes: POST /store/aid-posts, GET /store/aid-posts (with bbox filter), POST /store/aid-posts/:id/respond, PATCH /store/aid-posts/:id/fulfill. Include MikroORM migrations. The location should be fuzzed in the GET response the same way seller-location fuzzes coordinates.*

AI PROMPT — Unified Post Creator

> *Build a Post tab component in React Native + Tamagui that lets users create different types of community content. The screen shows a grid of post types: (1) 'List an Offering' — routes to product/service creation (MedusaJS seller flow), (2) 'Mutual Aid' — routes to AidPost creator (need or offer), (3) 'Post a Job' — routes to ShipmentBoardListing creator (Blackstar), (4) 'Start a Proposal' — routes to ProposalComposer (governance), (5) 'Launch a Campaign' — routes to collective-campaign creator (MedusaJS), (6) 'Share a Video' — routes to video upload flow (Matrix media). Each type has: an icon (solarpunk style), title, one-line description, and the color associated with its layer. The grid uses organic border-radius, hover/press states with glow effects, and staggered fade-in animations. Non-provider users see types 2, 4, and 6 (mutual aid, proposals, and videos are for everyone). Provider-only types show a subtle 'Provider' badge. When a general user taps a provider-only type, show a friendly upsell: 'Create a provider profile to list offerings' with a CTA button.*

PHASE 6 Weeks 23–26

Polish, Consent & App Store

Final polish: accessibility audit, performance optimization, comprehensive privacy/consent flows, App Store and Play Store submissions, offline mode, and the complete solarpunk animation system.

Tasks

#	Task	Owner	Status
1	Accessibility audit: screen reader support, minimum touch targets (44pt), color contrast (WCAG AA), dynamic type scaling	Design	**Critical**
2	Performance: lazy-load map layers, virtualize all lists, optimize MapLibre tile caching, reduce bundle size with tree-shaking	Dev	**High**
3	Comprehensive privacy settings screen: location consent management, data visibility controls, Matrix device verification, export/delete personal data	Dev	**Critical**
4	Offline mode: PMTiles cached map, SQLite entity cache, action queue (orders, votes, messages) that syncs when reconnected	Dev	**High**
5	Solarpunk animation system: Fibonacci spiral background decorations, sine wave contour animations, light ray conic gradients, organic marker floating using Reanimated	Dev	**High**
6	App Store submission: screenshots, app description, privacy policy, review guidelines compliance, TestFlight beta	DevOps	**Critical**
7	Play Store submission: signed APK/AAB, store listing, data safety form, internal testing track	DevOps	**Critical**
8	PWA deployment: configure service worker, manifest.json, web push notifications, deploy to app.freeblackmarket.com	Dev	**High**
9	Onboarding flow: animated walkthrough explaining the map, feed, chat, and community features; skip option for returning users	Design	**Normal**
10	Analytics: privacy-respecting usage metrics (no PII), error tracking via Sentry, performance monitoring	Dev	**Normal**

AI Prompts

AI PROMPT — Privacy Settings Screen

```
Build a comprehensive Privacy & Consent settings screen in React Native +
Tamagui for the BMC app. Sections: (1) LOCATION: current status (Sharing/Not
```

> *Sharing), toggle to enable/disable, display radius selector (Neighborhood/District/City/Region), 'Who can see my location' (Everyone/Followers/Nobody), explanation text for each option. (2) DATA VISIBILITY: profile visibility (Public/Community Only/Private), offerings visibility, activity visibility, wallet visibility. (3) MESSAGING: allow DMs from (Everyone/Followers/Nobody), read receipts toggle, typing indicators toggle, Matrix device list with verify/remove buttons. (4) ACCOUNT: export all my data (downloads JSON), delete my account (with confirmation and explanation of what gets deleted across MedusaJS + Blackstar + Blackout), linked accounts status. Each section has an organic card with the solarpunk surface style. Toggle switches use the green leaf color when active. Include an 'About Privacy' link at the top explaining the BMC privacy philosophy: consent-first, approximate-only location, E2EE messaging, no data selling, user-controlled visibility. All settings persist via gateway API calls that update preferences across all three backend systems.*

AI PROMPT — Solarpunk Animation System

> *Build a reusable animation system for the BMC app using react-native-reanimated and react-native-svg. Components: (1) FibonacciSpiral: SVG component that draws a golden spiral, slowly rotating (120s full rotation), used as background decoration on map and profile screens. Props: size, color, opacity, rotationDuration. (2) SineWaveBackground: animated SVG with 4-5 sine waves at different frequencies and amplitudes, gently oscillating. Each wave has a different color (alternating leaf green and solar gold) and a phase animation. Props: waveCount, baseAmplitude, baseFrequency, colors. (3) LightRay: a conic-gradient div that slowly shifts rotation and opacity, simulating golden-hour sun rays. Props: position, intensity, duration. (4) OrganicBlob: an animated border-radius component that continuously morphs between different organic blob shapes using interpolated border-radius values. Props: size, color, morphDuration, children. (5) PulseGlow: a component that adds a breathing glow effect around its children, using box-shadow animation. Props: color, intensity, duration. All animations should use useAnimatedStyle with withRepeat and withTiming/withSpring. Include a SolarpunkProvider that controls global animation speed (can reduce motion for accessibility). Show complete code with TypeScript types.*

AI Prompt Reference Index

All prompts in this document are designed to be used with Claude or similar AI coding assistants. Each prompt provides complete context about the existing codebase, the specific task, and the expected output format. Prompts reference real file paths, module names, and API patterns from the BMC ecosystem repositories.

Phase	Prompt Name	Systems Touched	Output Type
0	Monorepo Scaffold	blackstar_nav	Config + structure
0	MapLibre Replacement	RN Maps → MapLibre	Components + config
0	Martin Tile Server + PostGIS	Railway, Postgres	Dockerfile + config
0	Dark Solarpunk MapLibre Style	MapLibre GL	Style JSON
1	Gateway Spatial Feed	Hono, MedusaJS	API endpoint
1	MapCanvas + Clustering	MapLibre RN	React Native component
1	MedusaJS Location Extension	MedusaJS, PostGIS	Module + migration
1	Location Consent Flow	React Native	Component + context
2	Matrix SDK Integration	matrix-js-sdk, Synapse	Context + hooks
2	TikTok Video Feed	RN Video, Matrix	Screen component
2	E2EE Chat Comments Panel	matrix-js-sdk, Tamagui	Component
3	Blackstar Gateway Client	Blackstar, Hono	API client
3	Provider Profile Onboarding	All three backends	Multi-step flow
4	Port Governance Engines	Blackout, Matrix	Engines + hooks
4	Governance Map Layer	MapLibre, Matrix	Layer + sheet
5	Mutual Aid Module	MedusaJS, PostGIS	Module + API
5	Unified Post Creator	All backends	Tab component
6	Privacy Settings Screen	All backends	Settings screen
6	Solarpunk Animation System	Reanimated, SVG	Animation library

Each prompt is self-contained and can be given directly to an AI assistant with no additional context needed. They reference the actual repository structure, file paths, module names, API patterns, and technology versions from the BMC ecosystem. Adjust environment variable names and URLs for your deployment.